

DEPO & STOK ERP PORTALI — PROJE HAKİMİYETİ VE MÜLAKAT REHBERİ

Teknik Kod Analizi, Kritik Fonksiyonlar ve Sektörel Mimari Soru-Cevap Rehberi

1. BACKEND (.NET 8/9 C#) — En Çok Sorulacak 10 Kritik Fonksiyon

No	Fonksiyon / Yapı	Açıklama ve Mülakatta Verilecek Yanıt
1	AsNoTracking()	GET/Listeleme isteklerinde EF Core bellek izleyicisini devre dışı bırakır. %50 RAM tasarrufu sağlar ve sorgu süresini hızlandırır.
2	FindAsync(id)	Primary Key ile arama yapar. Önce EF Core RAM belleğine bakar, varsa SQL atmadan anında getirir; yoksa SQL çalıştırır.
3	SaveChangesAsync()	Bellekte yapılan tüm Add/Update/Delete işlemlerini biriktirip tek bir SQL Transaction paketiyle veritabanına kaydeder.
4	User.GetUserId()	JWT Token içerisindeki kullanıcı ID'sini güvenli çeken Extension metodudur. Hardcoded parsing ve '?? 0' bağımlılığını siler.
5	ToJson() / UpdateFromDto()	Entity ile DTO nesneleri arasındaki dönüşümü Controller dışına soyutlayan Extension Mapping yapısıdır.
6	StartsWith(text)	Full Table Scan yapan .Contains() yerine SQL B-Tree İndeksini kullanan (LIKE 'text%') yüksek performanslı arama metodudur.
7	BCrypt.Verify()	Kullanıcı parolalarını veritabanında düz metin saklamak yerine tek yönlü tuzlanmış (salted) hash ile doğrular.
8	AddScoped<T>()	Dependency Injection konteynerinde HTTP isteği başladığında nesne üretilip istek bittiğinde yok eden yaşam döngüsüdür.
9	AddJwtBearer()	API uç noktalarına gelen isteklerin başlığındaki dijital JWT imzasını doğrulayan güvenlik mekanizmasıdır.
10	[Authorize(Roles = ...)]	API Controller metodlarına sadece belirlenen yetki rolündeki kullanıcıların erişmesini sağlayan attribute'tur.

2. FRONTEND (REACT 19 + VITE) — En Çok Sorulacak 7 Kritik Fonksiyon

No	Fonksiyon / Yapı	Açıklama ve Mülakatta Verilecek Yanıt
1	axios.interceptors	Her HTTP isteğinin başlığına Authorization: Bearer <Token> bilgisini otomatik ekleyen merkezi HTTP istemcisi.
2	createContext / useContext	Kullanıcı oturumu, Dil (TR/EN) ve Para birimini Prop Drilling yapmadan tüm bileşenlerde paylaşan global state yönetimi.
3	useNavigate & useLocation	Sayfa yenilenmeden SPA (Single Page Application) yönlendirmesi yapan ve URL sorgu parametrelerini okuyan hook'lar.
4	<ProtectedRoute />	Giriş yapmamış veya yetkisi yetmeyen kullanıcıları otomatik /login sayfasına yönlendiren rotalama muhafızı.
5	useEffect()	Bileşen ekrana ilk yüklendiğinde (Mount) backend'den asenkron verileri çeken lifecycle hook'udur.
6	t('key') / useLanguage()	Seçili olan aktif dile göre (TR/EN) ilgili metin karşılığını arayüzde dinamik olarak gösteren i18n sistemidir.
7	defaultExpanded={false}	Yüzlerce akordiyonun aynı anda açılıp tarayıcıyı dondurmasını engelleyen varsayılan kapalı UI performans ayarıdır.

3. SEKTÖREL MİMARİ CEVAPLAR (Supervisor / Tech Lead Sorarsa)

• Soru: Neden Controller içinde DbContext kullanmadın?

Cevap: Katmanlı Mimari (Clean Architecture) ilkelerine uyduk. Controller'ları ince (Thin Controller) tutup tüm veritabanı sorgularını ve iş mantığını Servis katmanına (Fat Service) taşıdık.

• Soru: Veritabanındaki '1' rolü ne anlama geliyor?

Cevap: Kod içerisinde magic number kalmaması için RoleConstants.AdminId = 1 ve UserRoleEnum tanımladık. Koda bakan herkes 1'in Admin olduğunu anlar.

• Soru: Neden .Contains() yerine .StartsWith() kullandın?

Cevap: .Contains() SQL tarafında 'LIKE %text%' oluşturup Full Table Scan yapar. .StartsWith() ise 'LIKE text%' oluşturup veritabanının B-Tree indeksini kullanır.

4. MÜLAKAT İPUCU: BU PROJEDE ÖVÜNEBİLECEĞİN ÖZET

"Projede .NET 8 Web API ile katmanlı mimari (Clean Architecture) ve React 19 ile modüler bir ERP portalı geliştirdik. Performans için EF Core AsNoTracking ve B-Tree Index uyumlu arama sorguları kullandık; güvenlik tarafını ise JWT ve Role-Based Auth ile çözdük."